COLLEGE DEPLOYMENT

# Find My Class

## Deployment & Database Integration Guide

| | |
|---|---|
| **Production architecture** | **College data mapping** |
| **Security & operations** | **Faculty integration** |

Prepared for College IT, Database Administration, System Administration, and Project Coordination

**Version 1.0 | August 2026**

# Document Control

| Item | Value |
| --- | --- |
| Document | College Deployment and Database Integration Guide |
| Application | Find My Class |
| Version | 1.0 production foundation |
| Audience | College IT, DBA, system administration, project coordination |
| Source of truth | docs/college-deployment-guide.md |

The college-required name-and-phone lookup is restricted to low-sensitivity timetable and explicitly published contact information. It must not authorize grades, attendance, fees, documents, addresses, or other private student records.

# Table of Contents

1. Executive Overview 4

2. System Architecture 4

3. Technology Stack 5

4. Server Requirements 5

5. Data Model and Database Requirements 6

6. College Database Mapping 7

7. Integration Options 7

8. Faculty Integration 8

9. Authentication and Authorization 8

10. Security Requirements 9

11. Deployment Process 9

12. Environment Variables 11

13. Database Setup and Migrations 11

14. Updating Timetables 12

15. Faculty Management 12

16. Campus Map Setup 13

17. Backup and Recovery 13

18. Monitoring and Operations 14

19. Production Checklist 14

20. Troubleshooting 15

Release Verification Commands 15

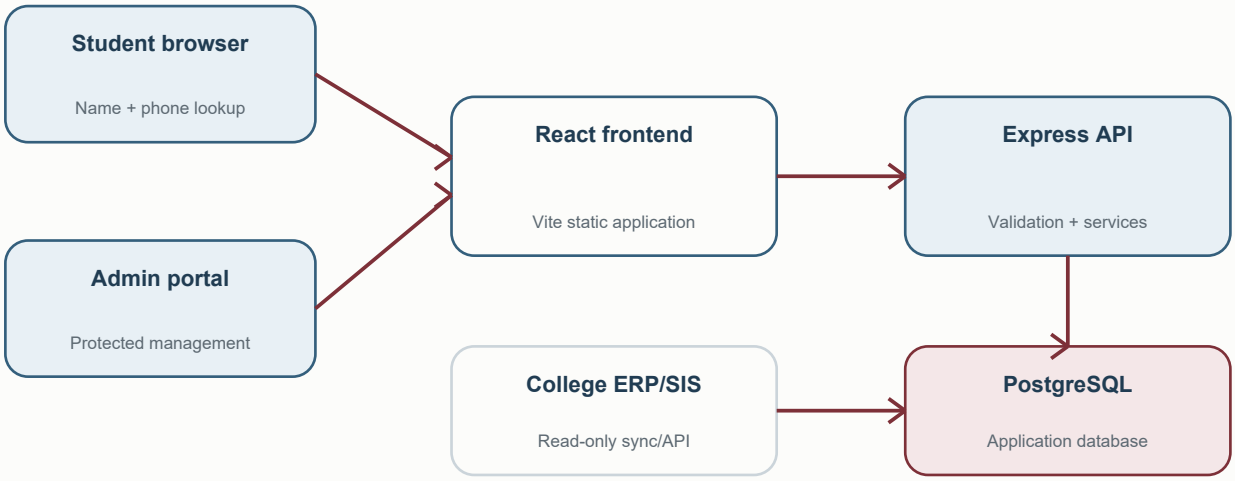
# 1. Executive Overview

Find My Class lets a student verify their record with their registered name and phone number, then view the current, next, daily, and weekly timetable for their assigned section. It presents classroom floor, wing, and room information; published faculty contacts; and a campus map with locally maintained pedestrian paths.

The protected admin portal manages students, subjects, classrooms, timetables, faculty contact details, class coordinators, student imports, and campus paths. Timetable image or text imports always produce an editable verification preview before saving.

The college-required student access method remains name plus phone number. This is appropriate only for the timetable and explicitly published contact information. It must not be reused to expose marks, fees, attendance, documents, or other sensitive student records.

# 2. System Architecture




The frontend may be hosted separately from the API, but a college-owned same-site domain is preferred, for example `timetable.example.edu` and `api.timetable.example.edu`. The API uses exact CORS origins and credentialed admin requests.

Key flows:

```
Student name + phone -> normalized identity -> phone HMAC lookup
-> section -> timetable + classrooms + derived faculty -> student result

Timetable teacher name -> normalized faculty match -> faculty directory
-> optional admin contact -> student Faculty page

Image/text upload -> validation -> OCR/parser -> day-grouped preview
-> admin correction -> timetable validation -> transaction -> faculty sync
```

# 3. Technology Stack

| Layer | Repository technology |
| --- | --- |
| Frontend | React 18, React Router 7, Vite 6 |
| Styling | Tailwind CSS 3, project design tokens, Geist and Space Grotesk fonts |
| HTTP client | Axios |
| Campus map | Leaflet, React Leaflet, Esri satellite tiles, OpenStreetMap street tiles |
| Backend | Node.js 20.12 or newer, Express 4 |
| Production database | PostgreSQL via `pg` |
| Local database | File-backed SQL.js SQLite-compatible database |
| Admin password hashing | bcryptjs, cost 12 for provisioned accounts |
| Admin session | Signed JWT stored in a Secure HttpOnly cookie, with CSRF header validation |
| Student phone lookup | HMAC-SHA256 keyed by `PHONE_LOOKUP_SECRET` |
| Student files | `@e965/xlsx` for CSV, XLS, and XLSX |
| Timetable OCR | Tesseract.js |
| Compression | Express compression for JSON/text responses over 1 KB |
| Tests | Node test runner, pg-mem, Vitest, Testing Library |

# 4. Server Requirements

## Minimum pilot environment

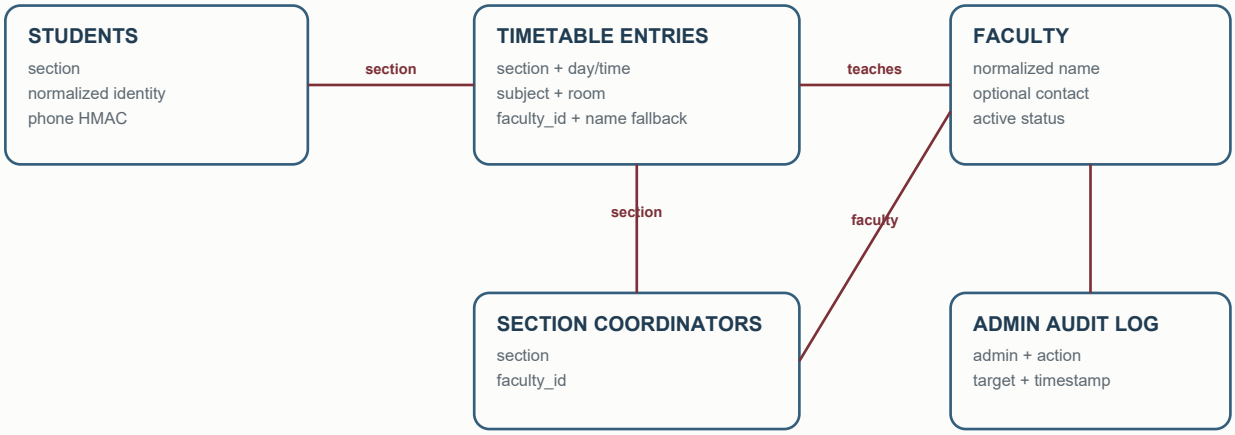
- 1 vCPU
- 1 GB RAM
- 10 GB SSD
- Ubuntu 24.04 LTS or equivalent supported Linux
- Node.js 20.12 or newer
- PostgreSQL 14 or newer
- Nginx or an equivalent reverse proxy
- Valid TLS certificate

## Recommended department/college environment

- 2 vCPU
- 2-4 GB RAM
- Managed PostgreSQL with automated backups
- Separate staging and production services
- Central log collection and uptime/error monitoring
- College-owned DNS and TLS

Capacity must be confirmed by staging load tests that reflect the college network and PostgreSQL latency. Local loopback benchmarks are not production capacity evidence.

# 5. Data Model and Database Requirements




Current persistent entities are:

| Entity | Purpose |
|---|---|
| `students` | Student identity, academic context, phone lookup hash, last four digits |
| `timetable_entries` | Section schedule rows, subject, faculty fallback name, room and session metadata |
| `subjects` | Admin-managed subject names |
| `classrooms` | Section/subject room fallbacks |
| `faculty` | Global faculty directory and optional published contact details |
| `section_coordinators` | Direct section-to-faculty coordinator assignment |
| `admins` | Hashed admin accounts and roles |
| `admin_audit_log` | Faculty contact administration events |
| `schema_migrations` | Applied migration versions |
| `timetable_seed_state` | Legacy bundled-data seed protection |

Logical college entities such as College, Department, Course, Academic Year, Semester, Section, and Timetable may remain in the college SIS and be synchronized into the application. They should only become separate application tables when the college data contract requires independent lifecycle management. This avoids speculative schema duplication.

Important constraints and indexes include unique university roll number, unique non-null student phone lookup hash, student identity lookup, student section, timetable section/day/start time, unique faculty normalized name, timetable faculty ID, and one coordinator per section.

Course, year, semester, and section remain validated application fields today rather than independent application tables. A college integration may map them from normalized ERP entities without forcing duplicate ownership into this database.

# 6. College Database Mapping

The table and column names below are examples only. The college DBA must map its actual schema to the application-required fields.

| Application-required field | Example college equivalent |
|---|---|
| student.id | STUDENT_MASTER.STUDENT_ID |
| student.name | STUDENT_MASTER.STUDENT_NAME |
| student.phone | STUDENT_MASTER.MOBILE |
| student.course | PROGRAM_MASTER.PROGRAM_NAME |
| student.year | STUDENT_ENROLMENT.CURRENT_YEAR |
| student.semester | STUDENT_ENROLMENT.SEMESTER |
| student.section | STUDENT_ENROLMENT.SECTION_CODE |
| student.active | STUDENT_MASTER.ACTIVE_FLAG |
| timetable.section | TIME_TABLE.SECTION_CODE |
| timetable.day | TIME_TABLE.DAY_OF_WEEK |
| timetable.startTime | TIME_TABLE.START_TIME |
| timetable.endTime | TIME_TABLE.END_TIME |
| timetable.subject | SUBJECT_MASTER.SUBJECT_NAME |
| timetable.faculty | TIME_TABLE.FACULTY_NAME or FACULTY_ID |
| timetable.room | TIME_TABLE.ROOM_NO |
| timetable.type | TIME_TABLE.SESSION_TYPE |
| faculty.id | EMPLOYEE_MASTER.EMPLOYEE_ID |
| faculty.phone | EMPLOYEE_CONTACT.PUBLISHED_MOBILE |
| classroom.building | ROOM_MASTER.BUILDING_NAME |
| classroom.roomCode | ROOM_MASTER.ROOM_CODE |

The integration must trim names, normalize section codes, map day/time values, reject inactive students, validate phone formats, and preserve a stable external identifier.

# 7. Integration Options

## Option 1: Application database import

The college exports CSV/XLS/XLSX files and admins import records into Find My Class. This is the simplest option when no ERP API exists. It is easy to audit but requires a defined update schedule.

## Option 2: Read-only ERP view

The college creates a restricted read-only view containing only the required fields. A synchronization job copies validated records into the application database. This avoids unrestricted ERP access and isolates timetable traffic from the ERP.

### Option 3: REST API integration

The college exposes authenticated student, section, timetable, faculty, and classroom endpoints. The application calls or periodically synchronizes from those endpoints. Use bounded timeouts, retries only for transient failures, and a staging contract test.

### Option 4: Scheduled ETL

An institution-managed process writes validated extracts to integration tables or files. This is suitable for nightly or hourly updates and creates a clear operational ownership boundary.

Do not write directly to the production ERP unless the college explicitly designs and authorizes that workflow. Prefer least-privilege read-only views, APIs, or synchronized integration tables.

---

## 8. Faculty Integration

Faculty names and contact details have separate sources of truth:

```
Timetable entry faculty name
-> conservative name normalization
-> unique faculty directory match
-> optional phone/designation/department managed by admin
-> section coordinator assignment
-> student Faculty page
```

The timetable determines who teaches a section. Repeated variants such as `Dr Amit Sharma`, `Dr. Amit Sharma`, and uppercase forms normalize to the same matching key. The system does not use fuzzy matching, so genuinely different names are not silently merged.

Phone number, designation, department, active status, and coordinator assignment belong to the faculty directory. A missing phone does not invalidate a timetable. The student page displays the faculty name with `Contact information not available.`

---

## 9. Authentication and Authorization

### Student access

Students enter their registered full name and Indian mobile number. The API normalizes both values, HMAC-hashes the phone number, and performs an indexed exact match. Full student phone numbers are not returned. Repeated failures are tracked by a hashed combination of name and phone, so one student's failures do not lock out everyone on shared college Wi-Fi.

The public test login is disabled unless `ENABLE_TEST_LOGIN=true`; production validation rejects that setting.

### Admin access

Admins use individually provisioned username/password accounts. Passwords are bcrypt-hashed. Sessions expire after 24 hours and are stored in Secure HttpOnly cookies. State-changing cookie requests require the matching CSRF token header.

Roles:

- `SUPER_ADMIN`: all current administration functions, including students/imports.
- `TIMETABLE_ADMIN`: timetable, subject, classroom, faculty, and map administration; student mutation APIs are denied.

# 10. Security Requirements

- Serve frontend and API exclusively through HTTPS.
- Permit only exact production frontend origins in `CLIENT_ORIGIN`.
- Store secrets in the platform secret manager or root-readable service environment file.
- Use different random values of at least 32 characters for JWT and phone lookup secrets.
- Never log full student phone numbers, passwords, tokens, or environment secrets.
- Restrict PostgreSQL network access and use a dedicated least-privilege database role.
- Keep the public response generic when student verification fails.
- Keep `ENABLE_TEST_LOGIN=false` and `LOAD_BUNDLED_DATA=false` in production.
- Restrict file types by extension, MIME, content signature, count, and configured byte limit.
- Keep Node.js and dependencies patched; fail deployment on production dependency advisories.
- Use a high-threshold edge/WAF abuse control if required, without a shared-IP student lockout.
- Rotate any credential that has appeared in screenshots, chat, logs, or source history.

# 11. Deployment Process

The current repository supports a Vercel frontend and a Node API behind Nginx/systemd. A college may host both services internally.

## Install and validate

```
git clone https://github.com/AdarshXtech/findmyclass.git
cd findmyclass
npm ci --prefix server
npm ci --prefix client
npm test --prefix server
npm run test:postgres --prefix server
npm test --prefix client
npm run build --prefix client
```

## Configure backend

```
cp server/.env.example server/.env
nano server/.env
npm run check:production --prefix server
npm run migrate --prefix server
npm run create-admin --prefix server
```

## Start backend with systemd

Example unit file `/etc/systemd/system/findmyclass.service`:

```
[Unit]
Description=Find My Class API
After=network-online.target

[Service]
Type=simple
WorkingDirectory=/var/www/findmyclass/server
EnvironmentFile=/var/www/findmyclass/server/.env
ExecStart=/usr/bin/npm run start:production
Restart=on-failure
RestartSec=5
User=findmyclass
Group=findmyclass
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now findmyclass
sudo systemctl status findmyclass --no-pager
```

## Nginx API proxy

```
server {
    listen 443 ssl http2;
    server_name api.timetable.example.edu;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 30s;
        client_max_body_size 6m;
    }
}
```

Set `TRUST_PROXY=1` when exactly one trusted reverse proxy is in front of Express.

## Frontend

Set `VITE_API_BASE_URL=https://api.timetable.example.edu`, then build:

```
npm run build --prefix client
```

Vercel must use repository root `.` with the root `vercel.json`, or project root `client` with `client/vercel.json`. Do not combine those configurations.

# 12. Environment Variables

| Variable | Required | Purpose / example format |
|---|---|---|
| `NODE_ENV` | Production | Must be `production` |
| `PORT` | Optional | API listener, default `5000` |
| `DATABASE_URL` | Production | PostgreSQL URL, stored as a secret |
| `DATABASE_PATH` | Local only | Local SQL.js database path |
| `CLIENT_ORIGIN` | Production | Comma-separated exact HTTPS frontend origins |
| `JWT_SECRET` | Production | Random admin session signing secret, 32+ characters |
| `PHONE_LOOKUP_SECRET` | Production | Separate random phone HMAC secret, 32+ characters |
| `TRUST_PROXY` | Behind proxy | Trusted proxy hop count, normally `1` |
| `JSON_BODY_LIMIT` | Optional | Express JSON limit, default `256kb` |
| `UPLOAD_LIMIT_BYTES` | Optional | Admin upload limit, default `5242880` |
| `STUDENT_LOOKUP_WINDOW_MS` | Optional | Per-identity failure window, default 15 minutes |
| `STUDENT_LOOKUP_MAX_FAILURES` | Optional | Failures before temporary identity lock, default 8 |
| `ENABLE_TEST_LOGIN` | Production | Must be `false` |
| `LOAD_BUNDLED_DATA` | Production | Must normally be `false` |
| `ADMIN_USERNAME` | Provisioning | Username consumed by `create-admin` |
| `ADMIN_PASSWORD` | Provisioning | 12+ character password consumed by `create-admin` |
| `ADMIN_ROLE` | Provisioning | `SUPER_ADMIN` or `TIMETABLE_ADMIN` |
| `STUDENT_ACCESS_RECORDS_JSON` | Legacy/dev | Transitional bundled roster phone mapping; avoid in production |
| `VITE_API_BASE_URL` | Frontend | Public API origin without `/api` |
| `API_PROXY_TARGET` | Local frontend | Local Vite API target |
| `API_PROXY_ORIGIN` | Local frontend | Optional development Origin header |

Never commit `server/.env`, `client/.env`, database files, logs, or production exports.

# 13. Database Setup and Migrations

1. Create an empty PostgreSQL database and restricted application role.
2. Set `DATABASE_URL` in `server/.env`.
3. Back up an existing database before every release.
4. Run `npm run migrate --prefix server`.
5. Run `npm run check:production --prefix server`.
6. Restart the API and verify `/api/ready`.

Migrations are additive and recorded in `schema_migrations`. Migration `001-production-foundation` creates the faculty directory, coordinator relation, audit log, role column, and timetable faculty link. It backfills existing timetable faculty names and legacy faculty contacts without dropping student or timetable data.

Rollback strategy: restore the pre-deployment database backup and deploy the previous application commit. Do not manually delete migration records while retaining their schema changes.

Bundled CSAI JSON data is development/transition data. It is loaded only when `LOAD_BUNDLED_DATA=true`. Normal production startup does not overwrite college-managed records.

# 14. Updating Timetables

Admins can add manually, edit, delete one entry, delete a complete timetable, insert between entries, shift selected entries, and import image/text data. Break and library periods do not require faculty; break entries do not require rooms.

Import flow:

1. Select course, year, and section.
2. Upload a PNG/JPEG/WEBP image or paste text.
3. Review extracted rows grouped by day.
4. Review detected faculty and missing contacts.
5. Correct subject, faculty, type, room, and time values.
6. Validate conflicts and classroom mappings.
7. Choose replace or merge.
8. Confirm save.

OCR output is never written directly to the timetable. Phone numbers are not required for faculty discovery or timetable validity.

# 15. Faculty Management

1. Open Admin -> Faculty.
2. Select the class/section.
3. Review unique names detected from the current timetable.
4. Select `Add phone number` or `Edit contact` for a detected person.
5. Enter an optional Indian phone number, designation, and department.
6. Assign `Coordinator` when appropriate.
7. Confirm replacement if another coordinator is assigned.

Clearing a contact removes published contact fields and coordinator linkage but preserves the faculty directory row and timetable name. Faculty edits are written to `admin_audit_log`. The current audit is intentionally small; a college SIEM integration may consume these records later.

# 16. Campus Map Setup

The college must provide verified:

- Campus points of interest and GPS coordinates
- Building names and aliases
- Accessible entrances
- Walkable outdoor path segments
- Closed/restricted areas

Destination data is maintained in `client/src/services/campusLocations.js`. The surveyed graph is in `client/src/services/campusPaths.js`. Admin Path Editor drafts are saved in that browser and exported as a JavaScript file; the exported file must be reviewed, placed into the repository, tested, built, and deployed.

Navigation remains manual:

```
Choose destination -> choose starting point/current location
-> preview -> Start Path
```

No route starts automatically. A straight-line fallback means the destination is not connected to the surveyed graph and the path network must be extended. Indoor or floor-level navigation is not claimed.

# 17. Backup and Recovery

Back up at least students, faculty, coordinators, timetables, classrooms, admin accounts, audit logs, and migration state.

Recommended policy:

- Managed PostgreSQL automated backups daily
- Point-in-time recovery/WAL retention where available
- Encrypted weekly export retained separately
- Backup before every migration or bulk import
- Quarterly restore exercise into an isolated environment
- Documented recovery point and recovery time objectives

Example logical backup and restore:

```
pg_dump --format=custom --no-owner --file=findmyclass.backup "$DATABASE_URL"
createdb findmyclass_restore_test
pg_restore --no-owner --dbname=findmyclass_restore_test findmyclass.backup
```

Never consider a backup verified until an authorized operator has restored it and checked student lookup, timetable retrieval, faculty contacts, and admin login.

# 18. Monitoring and Operations

- `GET /api/health`: process liveness; returns `{ "status": "ok" }`.
- `GET /api/ready`: executes a database query; returns HTTP 503 if unavailable.
- Collect systemd/Nginx logs centrally.
- Alert on repeated 5xx responses, readiness failure, high latency, CPU, memory, disk, and PostgreSQL connection exhaustion.
- Monitor certificate expiry, backup completion, and restore-test age.
- Preserve request IDs when reporting failures.
- Do not include student names or full phone numbers in metrics or logs.

Before a college-wide launch, load-test staging with realistic login arrival rates, sections, timetable sizes, PostgreSQL latency, and reverse-proxy configuration. Increase database pool size only after measuring queueing and database capacity.

# 19. Production Checklist

[ ] College owner and technical contacts assigned

[ ] Production and staging domains configured

[ ] HTTPS and automatic certificate renewal enabled

[ ] PostgreSQL database and least-privilege role configured

[ ] Pre-deployment backup completed

[ ] `npm run migrate --prefix server` completed

[ ] `npm run check:production --prefix server` passed

[ ] JWT and phone lookup secrets are separate and securely stored

[ ] Test login and bundled-data loading disabled

[ ] Individual admin accounts and roles created

[ ] Student records imported/synchronized

[ ] Timetables imported and manually verified

[ ] Faculty contacts approved for student display

[ ] Class coordinators assigned

[ ] Campus locations and path entrances verified

[ ] `/api/health` and `/api/ready` monitored

[ ] CORS tested against the exact frontend origin

[ ] Backup restore successfully tested

[ ] Staging smoke tests passed

[ ] Rollback owner and procedure confirmed

# 20. Troubleshooting

## Database connection failure

Run `curl -i https://api.example.edu/api/ready`, inspect `journalctl -u findmyclass`, confirm `DATABASE_URL`, firewall rules, TLS mode, credentials, and PostgreSQL connection limits.

## Frontend cannot reach API

Confirm `VITE_API_BASE_URL`, exact `CLIENT_ORIGIN`, HTTPS on both services, Nginx proxying, and the browser Network panel. A successful preflight must include the exact `Access-Control-Allow-Origin` and credentials headers.

## Student not found

Confirm the student is active, name normalization matches the college record, phone mapping was imported with the current `PHONE_LOOKUP_SECRET`, and the section has a timetable. Do not inspect or display the full phone through a public endpoint.

## Timetable missing

Check the student's section, `timetable_entries`, academic session, import mode, and admin verification result. Production startup no longer reloads bundled timetables automatically.

## Faculty number missing

The timetable can still be valid. Open Admin -> Faculty, select the section, choose the detected faculty name, and add an explicitly approved student-facing phone number.

## Image import failure

Confirm the file is a valid PNG/JPEG/WEBP under `UPLOAD_LIMIT_BYTES`, improve crop/contrast, or paste timetable text. Always review OCR output before saving.

## Map location permission denied

Use HTTPS, verify browser permission, or manually select a starting point. Location permission is optional and navigation never starts automatically.

## Admin repeatedly returns 401/403

Confirm the frontend sends credentialed requests, cookies are not blocked, the CSRF token is present for writes, system clocks are correct, and the admin role permits the operation. Prefer same-site college subdomains for reliable cookie handling.

# Release Verification Commands

```
npm test --prefix server
npm run test:postgres --prefix server
npm audit --omit=dev --prefix server
npm test --prefix client
npm run build --prefix client
npm audit --omit=dev --prefix client
```

Record the commit, migration list, test output, backup identifier, deploy time, operator, and rollback target for every production release.